

CARRERA: Maestría de Software

ASIGNATURA: Patrones de Diseño de APIs

NRO. PRÁCTICA:

04

TÍTULO PRÁCTICA: EVALUACIÓN DE ATRIBUTOS DE CALIDAD (ISO/IEC 25010)
MEDIANTE TESTING

OBJETIVO : Evaluar los atributos de calidad (ISO/IEC 25010)

INSTRUCCIONES

1. Validar la Modularidad y Capacidad de Prueba

2. Auditar el Comportamiento Temporal y Capacidad

3.

1. OBJETIVOS DE APRENDIZAJE

- **Validar la Modularidad y Capacidad de Prueba:** Demostrar el éxito del desacoplamiento arquitectónico (Patrón Repository) mediante el aislamiento completo de controladores HTTP utilizando dobles de prueba (Mocks).
- **Auditar el Comportamiento Temporal y Capacidad:** Evaluar la resiliencia perimetral de las reglas contractuales (Zod DTO) e identificar la degradación del rendimiento frente a escenarios de saturación transaccional incremental y concurrente.

2. PRE-REQUISITOS E INSTALACIÓN DEL ENTORNO

Para garantizar la compatibilidad con módulos nativos de **ECMAScript** ("type": "module") y la línea de desarrollo moderna de **Node.js v24+**, instalaremos herramientas que no dependan de las APIs obsoletas del compilador tradicional de TypeScript.

Instalación de Dependencias del Backend

Ejecuta el siguiente comando en la terminal dentro del directorio backend/:

```
npm install jest @types/jest ts-jest -D
```

3. DESARROLLO DE LA PRÁCTICA**FASE A: Pruebas Unitarias Aisladas (Jest) — Atributo: Mantenibilidad (Modularidad)**

El estudiante deberá implementar una prueba unitaria de caja blanca que verifique que el controlador funciona de manera agnóstica a la base de datos. Si el archivo de prueba requiere importar mongoose, la práctica será penalizada.

Configura el archivo backend/jest.config.js para dar soporte nativo a módulos de ECMAScript:

```
export default {
  preset: 'ts-jest/presets/default-esm',
  testEnvironment: 'node',
  verbose: true,
  clearMocks: true,
  testMatch: ['**/?(*.)+(spec|test).ts'],
  transform: {
    '^.+\\.tsx?$': ['ts-jest', { useESM: true }],
  },
  moduleNameMapper: {
    '^(\\.{1,2}/.*)\\.js$': '$1',
  },
}
```

```
},
};
```

2. Archivo de Prueba (src/modules/employee/controllers/employee.controller.spec.ts)

```
import { Request, Response } from 'express';
import { EmployeeController } from '../employee.controller';
import { IEmployeeRepository } from '../repositories/employee.repository.interface';

describe('□ Unit Test: EmployeeController (Mantenibilidad & Testabilidad)', () => {
  let controller: EmployeeController;
  let mockRepository: jest.Mocked<IEmployeeRepository>;
  let mockRequest: Partial<Request>;
  let mockResponse: Partial<Response>;
  let statusMock: jest.Mock;
  let jsonMock: jest.Mock;

  beforeEach(() => {
    // 1. Crear un Mock 100% aislado de la interfaz (Cero dependencia de Mongoose)
    mockRepository = {
      findAll: jest.fn(),
      findById: jest.fn(),
      create: jest.fn(),
      update: jest.fn(),
      delete: jest.fn(),
    };

    controller = new EmployeeController(mockRepository);

    // 2. Mockear los objetos del ciclo de vida de Express
    jsonMock = jest.fn();
    statusMock = jest.fn().mockReturnValue({ json: jsonMock });
    mockResponse = { status: statusMock };
  });

  it('Debería retornar un estado 200 y la lista de empleados de la abstracción', async () => {
    const fakeEmployees = [
      { nombre: 'Andrés Mendoza', cargo: 'Arquitecto', departamento: 'TI', sueldo: 4000 }
    ];

    // Configurar el comportamiento esperado de la abstracción
    mockRepository.findAll.mockResolvedValue(fakeEmployees);
    mockRequest = {};

    await controller.getEmployees(mockRequest as Request, mockResponse as Response);

    // Verificaciones asertivas del contrato
    expect(statusMock).toHaveBeenCalledWith(200);
    expect(jsonMock).toHaveBeenCalledWith(fakeEmployees);
    expect(mockRepository.findAll).toHaveBeenCalledTimes(1);
  });
});
```

Esta prueba inyecta ráfagas masivas de tráfico para medir la velocidad de rechazo perimetral del validador **Zod DTO** antes de comprometer los recursos del servidor de bases de datos.

Instala Artillery de forma global si no cuentas con el binario del sistema:
npm install -g artillery

2. Diseña el escenario transaccional mixto en la raíz de tu proyecto (stress-test.yml)

config:

target: "http://localhost:3000"

plugins:

ensure: {} # Falla automáticamente el proceso si no se cumplen los acuerdos de nivel de servicio (SLA)

phases:

- duration: 20

arrivalRate: 5

name: "1. □ Fase de Calentamiento (Tráfico Base)"

- duration: 30

arrivalRate: 15

rampTo: 50

name: "2. □ Fase de Saturación Máxima (Pico de Carga)"

ensure:

thresholds:

- "http.response_time.p99": 200 # El 99% de las transacciones debe procesarse en < 200ms

(Comportamiento temporal)

maxErrorRate: 1 # No se permite más del 1% de caídas del socket HTTP

scenarios:

- name: "Inundación transaccional perimetral (Payloads Mixtos Zod)"

flow:

Flujo A: Datos Correctos -> Evalúa la tubería completa refactorizada

- post:

url: "/api/v1/employees"

json:

nombre: "Andrés Mendoza"

cargo: "Software Architect"

departamento: "I+D"

sueldo: 4200

capture:

json: "\$.data._id"

as: "employeeId"

- think: 1

Flujo B: Datos Corruptos -> Evalúa el rechazo inmediato y óptimo del DTO Zod

- post:

url: "/api/v1/employees"

json:

nombre: "AI"

cargo: "Dev"

departamento: "TI"

sueldo: -500 # Campo ilegal

4. ENTREGABLES Y EVALUACIÓN (REPORTE DE INGENIERÍA)

El maestrante deberá compilar un informe técnico estructurado que responda a las siguientes métricas arrojadas por la suite de pruebas:

1. Análisis de Aislamiento (Evidencia Jest)

Demostrar que la ejecución de npm run test finaliza en verde de manera exitosa.

Pregunta de Control: ¿Por qué el uso del patrón de Inversión de Dependencias permite realizar la prueba unitaria del controlador sin necesidad de inicializar un contenedor Docker o instancia de MongoDB local?

2. Análisis del Percentil 99 (p99) y Rendimiento Colectivo (Evidencia Artillery)

Al finalizar la ejecución de artillery run stress-test.yml, extraiga del reporte de la consola las siguientes métricas y justifíquelas bajo la norma **ISO/IEC 25010**:

http.codes.201 **vs** http.codes.400: Verifique si el contador de rechazos de Zod coincide exactamente con el volumen de tráfico corrupto inyectado.

http.response_time.p99: Analice la latencia del percentil 99. Si el tiempo de respuesta creció de forma lineal o exponencial durante la Fase de Saturación Máxima, argumente técnicamente qué cuellos de botella síncronos dentro del Event Loop de Node.js pudieron haber provocado la degradación del rendimiento.

Docente / Técnico Docente:

Firma: _____

Resolución CS N° 076-04-2016-04-20